

Checklist de Requisitos

Tech Challenge Fase 3

MedAssist — Assistente Médico Inteligente com IA

FIAP Pós-Tech — Inteligência Artificial para Devs | Maio / 2026

Legenda:

✔ Implementado e funcionando

⚠ Implementado com ressalva

✖ Não implementado

1. Fine-tuning de LLM com Dados Médicos

1.1 Modelo, técnica e dados de treinamento

	Requisito do PDF	Onde está no projeto	Observação
✔	Fine-tuning de um LLM com dados médicos	fase2_finetuning/MedAssist_FineTuning_Colab.ipynb	Phi-3-mini-4k-instruct (3.8B) + QLoRA 4-bit via Unsloth
✔	Usar protocolos médicos / perguntas frequentes / laudos	medquad_dataset/ + data/medquad_ptbr.jsonl	MedQuAD: 5.274 pares Q&A de 12 fontes NIH/CDC/NCI
✔	Preprocessing, anonimização e curadoria dos dados	medquad_dataset/clean_medquad.py + fase1_dados/preprocess.py	Limpeza XML, filtros de qualidade, deduplicação, split 90/10
✔	Dataset de treinamento preparado	data/train.jsonl (4.747) + data/val.jsonl (527)	Formato JSONL — messages: system / user / assistant
✔	Execução do fine-tuning em ambiente com GPU	fase2_finetuning/MedAssist_FineTuning_Colab.ipynb	Google Colab T4 gratuito — ~45 min de treinamento
✔	Técnica de eficiência de memória (LoRA/QLoRA)	MedAssist_FineTuning_Colab.ipynb — QLoRA config	NF4 4-bit: VRAM de ~16 GB reduzida para ~6 GB

	Requisito do PDF	Onde está no projeto	Observação
✓	Hiperparâmetros documentados	relatorio_tecnico_medassist.docx — Seção 2	rank=16, alpha=32, lr=2e-4, batch_efetivo=16, 1 epoch

1.2 Adaptadores LoRA — Validação

	Requisito do PDF	Onde está no projeto	Observação
✓	Adaptadores LoRA gerados e armazenados	outputs/model/ (114 MB)	adapter_config.json + adapter_model.safetensors + tokenizer
✓	Validação automatizada dos adaptadores	fase2_finetuning/validate_adapters.py	28/28 verificações aprovadas: arquivos, config, tensores, tokenizer
✓	Avaliação qualitativa do modelo fine-tunado	outputs/colab_model_test.json	5/5 perguntas clínicas com respostas adequadas no Colab

2. Assistente Médico com LangChain + LangGraph

2.1 Pipeline RAG

	Requisito do PDF	Onde está no projeto	Observação
✓	Pipeline LangChain com LLM integrada	fase3_langchain/chains.py	RAG completo: embeddings MiniLM-L12 + FAISS + MedQuAD PT-BR
✓	Base de dados estruturada (vector store)	outputs/vectorstore/index.faiss	5.274 documentos indexados — similaridade cosine
✓	Contextualização das respostas com dados do corpus	fase3_langchain/chains.py → similarity_search()	Top-3 documentos mais relevantes injetados no prompt
✓	Modelo de embedding multilíngue	fase3_langchain/chains.py	paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers)
✓	Fontes exibidas em cada resposta	web_ui.html + fase4_seguranca/explainability.py	Perguntas MedQuAD consultadas visíveis no painel lateral

2.2 Fluxo LangGraph — Grafo de Estados

	Requisito do PDF	Onde está no projeto	Observação
✓	Fluxo de decisão automatizado (LangGraph)	fase3_langchain/graph.py	StateGraph com 6 nós e roteamento condicional

	Requisito do PDF	Onde está no projeto	Observação
✓	Nó de entrada — normalização da pergunta	<i>graph.py → node_entrada()</i>	Limpeza e normalização do input do usuário
✓	Nó de triagem — categorização automática	<i>graph.py → node_triagem()</i>	Detecta: emergência fora_escopo consulta
✓	Nó de emergência — bloqueio do LLM	<i>graph.py → node_emergencia()</i>	Resposta direta: SAMU 192 / Bombeiros 193 / CVV 188
✓	Nó de busca RAG	<i>graph.py → node_busca_rag()</i>	Recuperação de documentos do FAISS com similaridade
✓	Nó de geração — chamada ao backend LLM	<i>graph.py → node_geracao()</i>	Chama llm_backend.py com contexto RAG injetado
✓	Nó de validação — safety + disclaimer	<i>graph.py → node_validacao()</i>	Adiciona fontes, flags de segurança e aviso médico
✓	Roteamento condicional pós-triagem	<i>graph.py → route_after_triagem()</i>	emergência → node_emergencia demais → node_busca_rag

2.3 Backends de Inferência

	Requisito do PDF	Onde está no projeto	Observação
✓	Backend unificado configurável via .env	<i>fase3_langchain/llm_backend.py</i>	Mesma interface pública para OpenAI e Ollama
✓	MODO 1 — OpenAI API (gpt-4o-mini)	<i>.env → LLM_BACKEND=openai + OPENAI_API_KEY</i>	Rápido, sem GPU, sem instalação extra
✓	MODO 2 — Ollama + Gemma 3 local (gemma3:4b)	<i>.env → OPENAI_BASE_URL=http://localhost:11434/v1</i>	Sem API key, sem custo, funciona offline
✓	Troca de backend em tempo real	<i>web_ui.html — modal 'Configurar Backend'</i>	Sem reiniciar o servidor FastAPI

3. Interface Web Profissional

	Requisito do PDF	Onde está no projeto	Observação
✓	Interface web responsiva	<i>web_ui.html + web_app.py</i>	FastAPI serve HTML/CSS/JS na porta 8080
✓	Chat com bolhas diferenciadas por tipo	<i>web_ui.html</i>	Consulta (azul), emergência (vermelho), sistema (cinza)
✓	Painel de fontes e detalhes da resposta	<i>web_ui.html → painel lateral direito</i>	Mostra documentos MedQuAD consultados + score
✓	Badge de status do backend ativo	<i>web_ui.html → header</i>	Exibe: OpenAI API Ollama gemma3:4b

	Requisito do PDF	Onde está no projeto	Observação
✓	Atualização do backend pelo modal web	<code>web_ui.html + /api/config (FastAPI)</code>	Troca backend sem reiniciar — resposta imediata
✓	Disclaimer médico automático	<code>web_ui.html + graph.py → node_validacao()</code>	Exibido em todas as respostas
✓	Bolha de emergência com alerta visual	<code>web_ui.html (CSS + JS)</code>	Vermelho + ícone de alerta + SAMU 192 em destaque

4. Segurança e Validação

4.1 Safety Guard — Guardrails Clínicos

	Requisito do PDF	Onde está no projeto	Observação
✓	Limites: não prescrever sem validação humana	<code>fase4_seguranca/safety_guard.py</code>	Flag de prescrição + aviso obrigatório de revisão
✓	Guardrail de emergência médica	<code>fase3_langchain/graph.py → node_triagem()</code>	Bloqueia LLM — redireciona para SAMU 192 / Bombeiros 193
✓	Guardrail de saúde mental crítica	<code>fase4_seguranca/safety_guard.py</code>	Suicídio / automutilação → CVV 188
✓	Guardrail de fora do escopo	<code>graph.py → node_triagem() categoria: fora_escopo</code>	Saudações e tópicos não médicos → resposta padrão
✓	Testes dos cenários de segurança	<code>fase4_seguranca/safety_guard.py (main)</code>	5/5 cenários testados e aprovados

4.2 Logging e Auditoria

	Requisito do PDF	Onde está no projeto	Observação
✓	Logging detalhado para rastreamento e auditoria	<code>fase4_seguranca/logging_audit.py</code>	Gera logs/audit.log em formato JSON
✓	Campos auditados por interação	<code>logging_audit.py → log_interaction()</code>	timestamp, session_id, pergunta, backend, fontes, flags, tempo
✓	Arquivo de log persistente	<code>logs/audit.log</code>	JSON lines — uma entrada por interação

4.3 Explainability

	Requisito do PDF	Onde está no projeto	Observação
✓	Explainability — indicar fonte da informação	<code>fase4_seguranca/explainability.py</code>	Fontes MedQuAD + score de confiança por resposta

	Requisito do PDF	Onde está no projeto	Observação
☑	Fontes visíveis na interface web	web_ui.html → painel de detalhes	Perguntas MedQuAD exibidas ao lado de cada resposta

5. Pipeline de Dados — MedQuAD PT-BR

	Requisito do PDF	Onde está no projeto	Observação
☑	Dataset sugerido pelo TC (MedQuAD)	medquad_dataset/MedQuAD/ (NIH — GitHub)	Dataset público oficial — sugerido pelo enunciado do TC
☑	Aquisição do dataset	medquad_dataset/MedQuAD/ → git clone	~16.000 pares XML brutos de 12 fontes NIH/CDC/NCI
☑	Limpeza e filtragem de qualidade	medquad_dataset/clean_medquad.py	Parsing XML, filtros de tamanho, deduplicação → 8.653 pares EN
☑	Tradução automática para PT-BR	medquad_dataset/translate_ptbr.py	deep_translator (Google Translate) → 5.274 pares PT-BR
☑	Anonimização / dados públicos	MedQuAD (NIH — público)	Não contém dados de pacientes reais
☑	Exploração e análise do dataset	fase1_dados/explore_dataset.py	outputs/explore_report.txt gerado
☑	Validação de qualidade dos dados	fase1_dados/validate_data.py	outputs/data_quality_report.txt — APROVADA
☑	Dataset documentado	medquad_dataset/README.md	Pipeline completo: fontes, filtros, tradução, estatísticas

6. Código e Organização

	Requisito do PDF	Onde está no projeto	Observação
☑	Projeto Python modularizado	Raiz do projeto	5 fases independentes: dados, FT, langchain, segurança, avaliação
☑	Ambiente virtual documentado	requirements.txt + README.md	pip install -r requirements.txt — dependências explícitas
☑	Documentação detalhada	README.md	Instalação, 2 modos, fluxo sequencial, troubleshooting
☑	Relatório técnico	relatorio_tecnico_medassist.docx	13 seções — arquitetura, fine-tuning, RAG, segurança, avaliação
☑	Avaliação do sistema	fase5_avaliacao/ (benchmark.py + generate_report.py)	Métricas ROUGE, BLEU, F1 + relatório técnico final

	Requisito do PDF	Onde está no projeto	Observação
✓	Configuração via .env	.env.example	Template com MODO 1 (OpenAI) e MODO 2 (Ollama)
✓	.gitignore configurado	.gitignore	.env e __pycache__ excluídos

7. Entregáveis

7.1 Repositório Git

	Requisito do PDF	Onde está no projeto	Observação
✓	Código-fonte completo	Todos os .py + web_ui.html + config	6 módulos Python + interface web
✓	Pipeline de fine-tuning	fase2_finetuning/MedAssist_FineTuning_Colab.ipynb	Notebook Colab completo com QLoRA
✓	Integração com LangChain	fase3_langchain/chains.py + assistant.py	RAG pipeline + assistant interface
✓	Fluxos do LangGraph	fase3_langchain/graph.py	StateGraph com 6 nós documentados
✓	Dataset anonimizado / sintético	data/medquad_ptbr.jsonl	5.274 pares públicos NIH — sem dados de pacientes
✓	Adaptadores LoRA do fine-tuning	outputs/model/ (114 MB)	adapter_model.safetensors — 28/28 validações OK
✓	.env.example	.env.example	Template com MODO 1 e MODO 2 comentados
✓	.gitignore	.gitignore	.env e caches excluídos

7.2 Relatório Técnico

	Requisito do PDF	Onde está no projeto	Observação
✓	Explicação do processo de fine-tuning	relatorio_tecnico_medassist.docx — Seção 2	Parâmetros QLoRA, dataset, validação 28/28
✓	Descrição do assistente médico criado	relatorio_tecnico_medassist.docx — Seções 3 e 4	LangChain, RAG, LangGraph, backends
✓	Diagrama do fluxo LangChain / LangGraph	relatorio_tecnico_medassist.docx — Seção 1	Fluxo completo dos 6 nós + integração dos componentes
✓	Avaliação do modelo e análise dos resultados	relatorio_tecnico_medassist.docx — Seção 8	Validação dos componentes + métricas + decisões
✓	Estratégias de segurança e guardrails	relatorio_tecnico_medassist.docx — Seção 5	Safety Guard, logging, explainability

	Requisito do PDF	Onde está no projeto	Observação
✔	Pipeline de dados documentado	<code>medquad_dataset/README.md</code>	Etapas, filtros, tradução e estatísticas

7.3 Vídeo de Demonstração

	Requisito do PDF	Onde está no projeto	Observação
⚠	Vídeo de até 15 minutos	— (pendente gravação)	Demonstração da interface web + funcionalidades
⚠	Treinamento e funcionamento da LLM personalizada	— (pendente gravação)	Fine-tuning no Colab + adaptadores LoRA
⚠	Execução de fluxo automatizado	— (pendente gravação)	LangGraph: triagem → RAG → geração → validação
⚠	Resposta a perguntas clínicas contextualizadas	— (pendente gravação)	Demonstrar fontes MedQuAD nas respostas
⚠	Logs e validação das respostas	— (pendente gravação)	Exibir logs/audit.log + painel de detalhes

8. Resumo Final

Categoria	Resultado
Fine-tuning de LLM (Phi-3-mini + QLoRA)	✔ 100% implementado — 28/28 validações OK
Dataset MedQuAD PT-BR processado	✔ 5.274 pares Q&A — limpeza + tradução + split
Pipeline RAG com LangChain + FAISS	✔ 5.274 documentos indexados — top-3 por consulta
Fluxo LangGraph (6 nós)	✔ Triagem → emergência RAG → geração → validação
Segurança — Safety Guard	✔ 5/5 cenários aprovados (emergência, prescrição, escopo)
Logging e Auditoria	✔ logs/audit.log JSON — todas as interações
Explainability — fontes por resposta	✔ Fontes MedQuAD + score de confiança
Interface Web FastAPI	✔ Chat + fontes + badge de backend + troca em tempo real
Dois backends configuráveis	✔ MODO 1 (OpenAI) + MODO 2 (Ollama gemma3:4b)
Código modularizado + README completo	✔ 5 fases independentes + instruções sequenciais
Relatório técnico	✔ 13 seções — docx completo
Vídeo de demonstração	⚠ Pendente — roteiro pronto

✔ Implementados: 49 itens ⚠ Pendentes: 5 itens (vídeo)

FIAP Pós-Tech — Inteligência Artificial para Devs | Tech Challenge Fase 3 | Maio / 2026